Department of Electrical and Computer Engineering
North South University

Senior Design Project

Facial Movement Based Mouse Cursor Operation for Differently Abled People through Camera

Md Moin Uddin **ID:1711132042**

Nasim Ahmed Bulbul **ID:1711608042**

Shaiqa Sharmeen **ID:1711713042**

Faculty Advisor:

Zunayeed Bin Zahir

Lecturer

Department of Electrical and Computer Engineering

Spring 2021

Declaration

This is to declare that no part of this report or the project has been previously submitted elsewhere for the fulfillment of any other degree or program. Proper acknowledgement has been provided for any material that has been taken from previously published sources in the bibliography section of this report.

.....
Md Moin Uddin
Department of Electrical and Computer Science
North South University
Dhaka, Bangladesh

.....
Nasim Ahmed Bulbul
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

.....
Shaiqa Sharmeen
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Approval

The Senior Design Project entitled "**Facial Movement Based Mouth Cursor Operation for Differently Abled People through Camera**" by Md Moin Uddin (ID:1711132042), Nasim Ahmed Bulbil (ID:1711608042) and Shaiqa Sharmeen (ID:1711713042) has been accepted as satisfactory and approved for partial fulfillment of the requirement of BS in CSE degree program on May, 2021.

Supervisor's Signature

Zunayeed Bin Zahir
Lecturer
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Department Chair's Signature

Dr. Rezaul Bari
Associate Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Acknowledgement

First of all, we would like to express our profound gratitude to our honorable course instructor, **Zunayeed Bin Zahir,** for his constant and meticulous supervision, valuable suggestions, his patience and encouragement to complete the thesis work.

We would also like to thank the ECE department of North South University for providing us with the opportunity to have an industrial level design experience as part of our curriculum for the undergraduate program.

Finally, we would like to thank our families and everybody who supported us and provided with guidance for the completion of this project.

Abstract

In recent years, new methods and models of Human Computer Interaction application are being developed. They are based on interaction with computer through head, eyes, facial expression, hand movements, voice recognition etc. Here we present a human computer interaction application that eases difficulties for differently abled people to control the mouse cursor of a computer through a camera. No wearables or no biosensor-based devices needed. The main idea is based on facial parts localization with help of a trained model, calculating movement estimators of these parts which can be used to operate a mouse. This will help people suffering from Quadriplegia, Multiple Sclerosis, Cerebral Palsy or other types of temporary or permanent accidental injuries that generally affects limb (arms or legs) movements.

Table of Contents

Chapter 1 10
Project Overview 10
1.1 Introduction 12
1.2 Facial Movements 12
1.3 Our Proposed Project 12
1.3.1 Brief Description of the Project Idea 13
1.3.2 Facial Landmarks and its Basics 13
1.3.3 Difficulty 14
1.4 Motivation 14
1.5 Summary 15
Chapter 2 16
Related work 16
2.1 Introduction 17
2.2 Existing Wearables and Assistive Device 17
2.3 Research and Publications 18
2.3.1 One millisecond face alignment with an ensemble of regression trees 19
2.3.2 Real-Time Eye Blink Detection using Facial Landmarks 19
2.3.3 300 Faces in-the-Wild Challenge: The first facial landmark localization Challenge 20
2.3.4 Mouse Handling Using Facial Features 20
2.3.5 Controlling Mouse Motions Using Eye Blinks, Head Movements and Voice Recognition 20
2.4 Summary 21
Chapter 3 22
Theory 22
3.1 Introduction 23
3.2 Detect Eyes, Nose, Mouth, Jaw from Facial Landmarks Using Trained Model 23
3.3 Calculation to Detect Action for Controlling Mouse 26
3.4 Summary 29
Chapter 4 30
Structure of the System 30
4.1 Introduction 31
4.2 Control Actions 31
4.2.1 Relation of Control Action with Theoretical Calculation 32
4.3 Workflow 33
4.3.1 Flowchart 33
4.4 Summary 34
Chapter 5 35

Technologies and Libraries 35
5.1 Introduction 36
5.2 Programming Language 36
5.3 Libraries 36
5.4 Trained Model 38
5.5 Dataset 38
5.6 Summary 39
Chapter 6 40
Technical Module I: Detecting Facial Features 40
6.1 Introduction 41
6.2 Static Images 41
6.2.1 Loading and Presenting an Image 41
6.2.2 Face Recognition 42
6.2.3 Identifying Facial Parts 42
6.2.4 Predicting All Landmarks 43
6.3 Real Time Video Sequence 44
6.4 Challenges 44
6.5 Testing Images 46
6.6 Summary 48
References 49
Appendix A 50

List of Figures

Fig No.	Figure Caption	Page No.
1.1	Predicted landmarks on face	13
1.2	Handsfree computer control	14
2.1	GlassOuse assistive head mouse control	17
3.1	Ensemble of regression tree	23
3.2	68 2D facial landmarks	24
3.3	Points of eye for EAR calculation	25
3.4	Points of mouth for MAR calculation	26
3.5	Comparison of nose point with anchor point	27
4.1	Control Action	30
4.2	Flowchart of the system	32
5.1	Python logo	35
5.2	OpenCV logo	36
5.3	NumPy logo	36
5.4	Dlib logo	37
6.1	Loading an image	40
6.2	Face recognition	41
6.3	Facial parts localization	42
6.4	Predicting 68 landmarks on face	43
6.5	Two portraits in different light condition	44
6.6	Low light detection problem	44
6.7	Test images	45-47

List of Tables

Tab No.	Table Caption	Page No.
3.1	Index representation of total 68 facial points	25
4.1	Control action estimator	31

Chapter 1
Project Overview

1.1 Introduction

Application of personal computer (PC) is widely outspread over all the areas such as communication, production, entertainment, research and education. Some severe disabling diseases of the brain and spinal cord result in total or partial loss of limb movements. Accidental injuries cause temporary or permanent restricted muscle movements. Some computer users do not have enough use of their hands and arms to operate a standard keyboard or mouse. Generally, they face a huge challenge on using the computers for their day-to-day activities. In this paper we present a Human Computer Interaction (HCI) application that will ease the difficulties for differently abled people. They will be able to operate a standard mouse through a camera with their own facial movements. No wearables and biosensors are needed.

1.2 Facial Movements

Facial movement is basically the muscle movement of different parts or regions of the face beneath skin. Eye blinking (closing and opening of the eyelid), yawning (stretching of eardrums), pouting (pushing lips forward), squinting (looking with partially closing eye) etc. actions are done by facial movements. They are primarily used for nonverbal communications; social and emotional message convey between humans and most other mammals and animals do so. However, we can use these actions for interaction with computers in a Human Computer Interaction system.

1.3 Our Proposed Project

We aim to build a simple, low-cost Human Computer Interaction application that helps differently abled people in mouse operation with their facial movements through a standard camera.

1.3.1 Brief Description of the Project Idea

Our project idea is heavily influenced by facial landmark localization. Detecting facial parts or regions from image or video, calculating features etc. are also influential. We will discuss these things in details later. Here we want to lighten up the idea in short.

A standard camera takes the input frames from the user when he or she starts to operate the computer. For personal computer, we propose a standard web camera. For laptop, generally web camera remains integrated. The input frame is mainly a video sequence. We can detect the user's face from the video sequence. Our trained model then converts the face into 68 2D facial landmarks. The facial landmarks are the representation of nose, mouth, jaw and eyes of the face. These 68 points are actually index that can be accessed like an array or a matrix. Using the array or the matrix we can calculate facial movements that means eye blink, mouth opening/closing etc. That will trigger the action that controls the mouse cursors. We will propose some actions to do this.

1.3.2 Facial Landmarks and its Basics

Facial landmarks are defined as the detection and localization of certain points characteristics on the face. Commonly used landmarks are the eyes, nose, mouth, eyebrow, jaws, chin etc. It plays arguably the important role as an intermediary step for many subsequent face processing operations that ranges from biometric recognition to the understanding of mental states. During the last decades we notice a wealth of scientific research in computer vision for the problem of facial landmark points localization. It is now also using in human computer interaction application along with other countless application. [1]

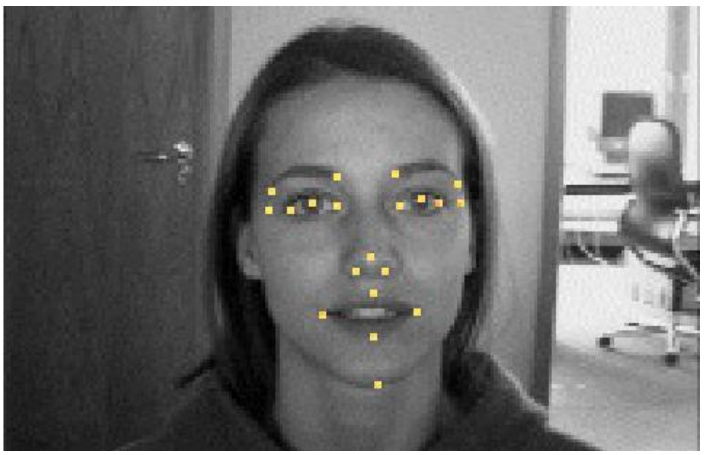


Fig 1.1: Predicted landmarks on face

1.3.3 Difficulty

One of the complex challenges is low light. As standard cameras or webcams are used it will be very difficult to find face and detecting the landmarks in low light. Another challenge we will be facing is to make the system latency free. Making it super real time would a bit difficult. Finding landmarks on the faces with facial hair or burnt faces or abnormal faces will be really challenging. Small blinks and short blinks happen real quick detecting those blinks would be a challenge.

1.4 Motivation

Human nature demands independence, similar is the case with differently abled physically handicapped people. Technology is now rapidly widening to cut off human sufferings. If we can design interfaces for them to interact with computation systems, they will be added to information system. Through this project, our objective is to design, develop and build a system that will provide a useful and efficient means of using computer for the differently abled people with ease. They will operate the mouse cursor without hands or legs but only using their facial movements.

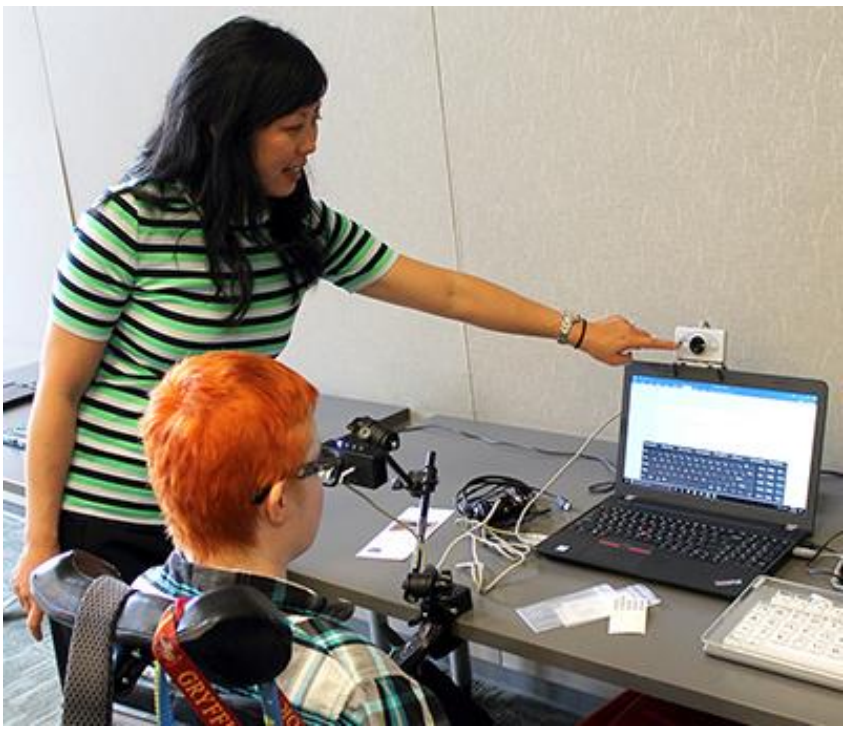


Fig 1.2: Handsfree computer control

1.5 Summary

In this chapter, we have briefly described the basics of facial movements, the brief description of our idea, facial landmarks and its basics. We have described the difficulties, motivation to design of the proposed system, what motivated us to design and build this system, and our accomplishments in here. The following chapters describe the related works, theory, structure, software component etc.

Chapter 2
Related work

2.1 Introduction

The existing works related to facial movement-based or hands-free mouse control will be discussed in this chapter. We will talk over wearables, sensors based assistive device and research papers. Wearables and devices are primarily based on the subject eye movement or subject head movement. However, these devices are really expensive and requires pre learning before using the system. Another drawback is it requires to wear a relatively complex piece of equipment either on their head, face etc. In research papers we see works containing image-based algorithm and feature based algorithm both actually produce vision based human computer interaction system. We hope these human computer interface systems will outperform the high-cost equipment in terms of expense.

2.2 Existing Wearables and Assistive Device

Several wearables and assistive device are in the market and this market is continuously growing among those three are listed below.

GlassOuse V1.3 - Head Mouse Control: GlassOuse allows someone with limited hand control opportunity to use computers and other personal devices by simply moving their head via Bluetooth. The device can control any mouse driven software. It can also control Android devices. GlassOuse is charged with a standard micro–B USB cable, and the battery runs up to 150 hours for full day use. It works fine but it comes with a price tag of **599.00 USD.** [2]

Tobii Windows Control 2: It is a gaze enabled computer access. Users simply look at the computer screen and control the mouse cursor with eye movement. It is compatible with most of the eye tracking hardware. This device costs a whopping **1511 USD.** [3]

TrackerPro 2: The TrackerPro 2 is a head mouse. It provides reliable hands-free mouse control for the people who are unable to use a traditional computer mouse. An individual can use almost all kind

of software, surf the internet, read a digital document, send an email or just about anything else that can be done on a computer, tablet, or phone. A small reflective dot is placed on the forehead, nose, or glasses of the user and small head movements control the mouse cursor movement. It connects to a computer with USB type A port. The price of the TrackerPro 2 is **995 USD.** [4]



Fig 2.1: GlassOuse assistive head mouse control

2.3 Research and Publications

In recent years there has been an expanding interest in developing natural interaction between human and computer. Multiple studies for human-computer interaction in universal computing is introduced. We have gone through some research papers for ideas and implementation.

2.3.1 One millisecond face alignment with an ensemble of regression trees

This paper solves the problem of face alignment for a single image. For this, facial landmark localization is necessary. Generally predicting facial landmark is a regression problem. Here an ensemble of regression trees used to estimate the facial landmark positions directly from a sparse subset of pixel intensities, achieving super real-time performance with high quality predictions. It presents a general framework based on gradient boosting for learning an ensemble of regression trees that optimizes the sum of square error loss and naturally handles missing or partially labelled data. Based on this work, facial landmarks are predicted using various face databases later. [5]

2.3.2 Real-Time Eye Blink Detection using Facial Landmarks

This paper proposed a real-time algorithm to detect eye blinks in a video sequence from a standard camera. Recent landmark detectors, trained on in-the wild datasets exhibit excellent robustness against a head orientation with respect to a camera, varying illumination and facial expressions. Accurately predicted landmarks are enough to reliably estimate the level of the eye opening. The proposed algorithm therefore estimates the landmark positions, extracts a single scalar quantity – eye aspect ratio (EAR) – characterizing the eye opening in each frame. Finally, an SVM classifier detects eye blinks as a pattern of EAR values in a short temporal window. The simple algorithm outperforms the state-of-the-art results. [6]

2.3.3 300 Faces in-the-Wild Challenge: The first facial landmark localization Challenge

This paper showed the first standardized benchmark for facial landmark localization. The main goal of this challenge is to compare the performance of facial landmark prediction of different methods using same evaluation protocol on face datasets. The challenge was held in con-junction with the international conference on computer vision 2013, Sydney, Australia. Facial datasets containing both constrained and unconstrained conditions have been used here. [7]

2.3.4 Mouse Handling Using Facial Features

The paper presented a system for eye blink detection and head movements for handling mouse. The proposed system provides a comprehensive solution to control computer mouse cursor movement with eye-blinks and head movement for people with special needs. The user may perform the clicking of mouse through eye-blinks. For detection of face candidates our basic strategy is use of Six-Segmented Rectangular Filter (SSR) along with a support vector machine (SVM) for face verification. This image-based algorithm with a Machine Learning classifier is computationally complex. [8]

2.3.5 Controlling Mouse Motions Using Eye Blinks, Head Movements and Voice Recognition

This work is a multimodal approach using eye blinks, head movements and voice recognition to create a hands-free interface between human and computer. It uses a typical webcam to capture an input image. Controlling of mouse cursor is obtained by face movement as moving face up, down, left and right and mouse events are controlled through eye blinks and voice. To perform these operations

different algorithms like Haar Cascade algorithm, Template Matching and Hough transformation are used. This is also an image-based system that adds complementary voice recognition features. [9]

2.4 Summary

The existing work related to facial movement based or handsfree mouse cursor control that we found useful have been briefly described in this section. The next chapter elaborates more on the theoretical part of our project.

Chapter 3
Theory

3.1 Introduction

The details of the theory of our system are discussed in this chapter. The theoretical explanation is divided into two sections sequentially.

1. Detect eyes, nose, mouth, jaw from facial landmarks using trained model
2. Calculation to Detect Action for Controlling Mouse

3.2 Detect Eyes, Nose, Mouth, Jaw from Facial Landmarks Using Trained Model

Facial landmark localization is almost similar to a shape prediction problem. A shape predictor attempts to localize salient parts of the face along with the shape in a given input image. In the context of facial landmarks, detecting facial landmarks is therefore a two-step process:

Step 1: Detect the face on the image

Step 2: Localize the key facial points or parts on the face.

There are several ways for step 1 like Open CV's built in Haar cascade, pre trained HOG + SVM classifier-based detector, deep learning or machine learning based algorithm etc. The algorithms used to detect the face in the image less matters rather than obtaining the facial coordinates of the face.

For Step 2, There are a variety of facial landmark detectors, but we want to use a pre-built trained model. For trained model we will use dlib's pre built model which is state of the art implementation of accurate prediction of 68 2D facial landmarks.

The dlib's pre-trained facial landmark detector is used to localize the location of 68 (x, y)-coordinates that map to facial structures on the face. The shape predictor algorithm implemented in the dlib library

built with an ensemble of regression tree techniques. What is Ensemble Modelling? Basically, Ensemble modeling is a process where multiple different and diverse models are trained to predict an outcome. It can be done by using many different modelling algorithms or using different training data sets. The motivation of using this model is to simply reduce the error of a prediction. [10] So, an ensemble model of a regression tree means multiple regression trees are working. Taking features of input images and after passing it through an ensemble of regression trees refines the predicted landmarks to improve accuracy. And by doing this, this shape predictor actually runs like super-real time.

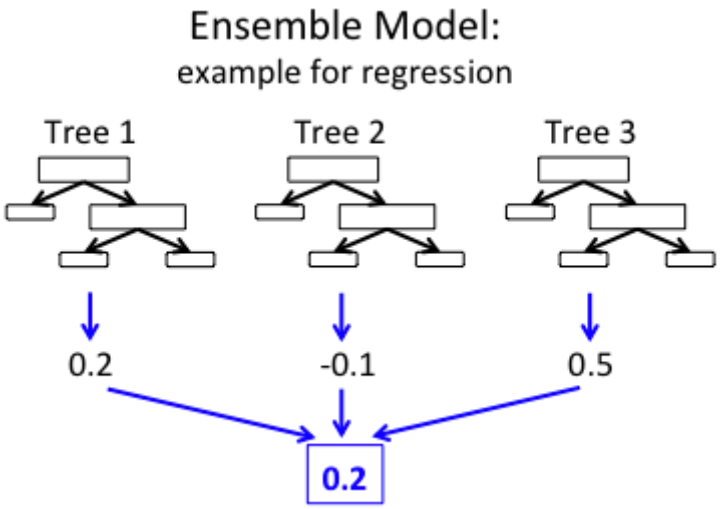


Fig 3.1: Ensemble of regression tree

The indexes of the 68 coordinates on a plain paper is shown below:

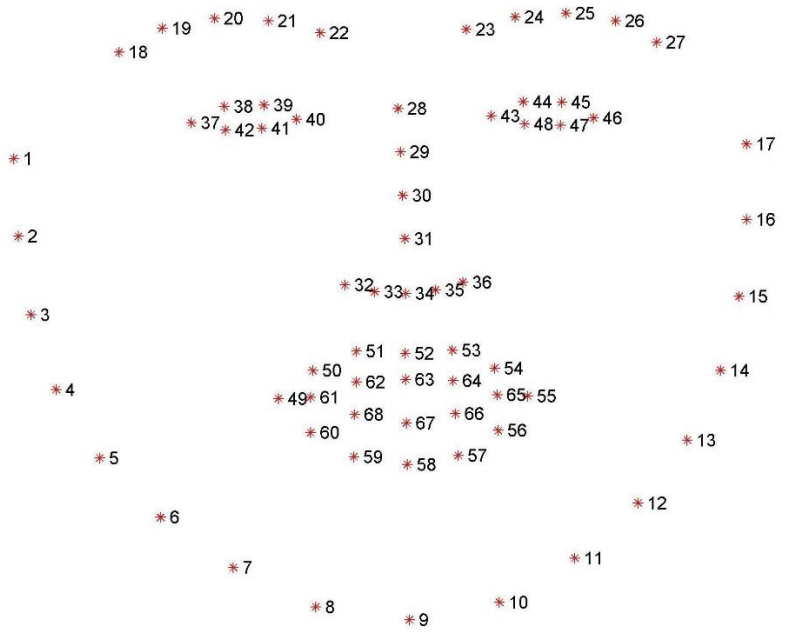


Fig 3.2: 68 2D facial landmarks

From the above figure we can see 37-42(facial points) represents the right eye, 43-48(facial points) represents the left eye, 49-68 (facial points) represents the mouth, 28-36 (facial points) represents the nose of the face.

Region	Start index	End index
Jaw	1	17
Right eyebrow	18	22
Left eyebrow	23	27
Nose	28	36
Right eye	37	42
Left eye	43	48
Mouth	49	68

Table 3.1: Index representation of total 68 facial points

3.3 Calculation to Detect Action for Controlling Mouse

In this section we will discuss about developing relevant estimators that will help us to calculate certain actions.

Eye Aspect Ratio

Eye aspect ratio is the estimator of eye opening and closing. It is a scalar quantity that calculates the ratio of the height and width of the eyes. Different values of Eye Aspect Ratio indicate different actions like eye blink (involuntary eye opening and closing), squint (looking with partially closed eye), wink (eye opening and closing quickly). [6]

Eye Aspect Ratio (EAR) = ||P2-P6|+|P3-P5|| / 2*||P1-P4||

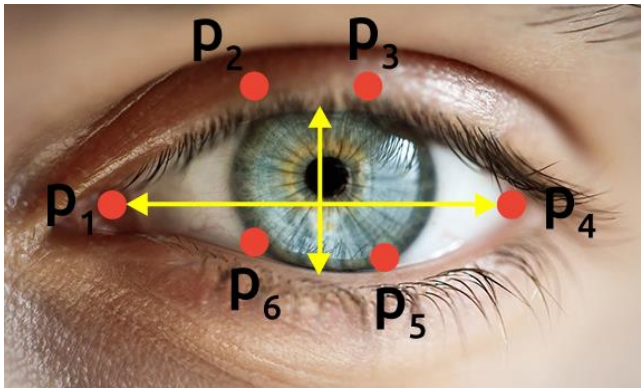


Fig 3.3: Points of eye for EAR calculation

From the image eye closed state

Distance (P2, P6) = 0

Distance (P3, P5) = 0

So, EAR = 0+0/2*||P1-P4|| = 0

Eye opened state

Distance (P2, P6) = C1 (const)

Distance (P3, P5) = C2 (const)

So, EAR = C1+C2/2*||P1-P4|| = C3 [Not zero]

Mouth Aspect Ratio

Mouth aspect ratio is the ratio of height between lips and width of the lips. It is the indicator of mouth opening and closing. Different values of Mouth Aspect Ratio indicate different actions of mouth opening and closing. [11]

Mouth Aspect Ratio (MAR) = ||P2-P8|| + ||P3-P7|| + ||P4-P6|| / 2*||P1-P5||

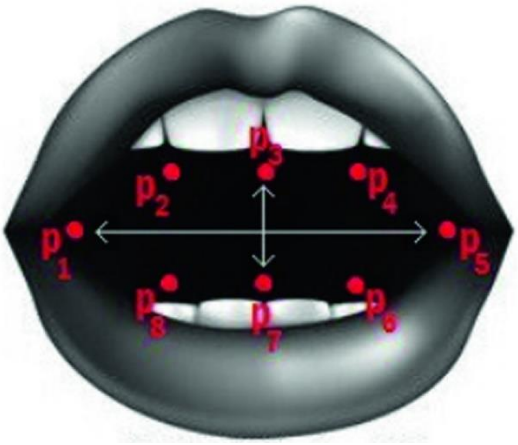


Fig 3.4: Points of mouth for MAR calculation

From the image mouth closed state

Distance (P2, P8) = 0

Distance (P3, P7) = 0

Distance (P4, P6) = 0

So, MAR = 0+0+0 / 2*||P1-P5||=0

Mouth opened state

Distance (P2, P8) = C1

Distance (P3, P7) = C2

Distance (P4, P6) = C3

So, MAR = C1+C2+C3 / 2*||P1-P5||= C4 [Not zero]

Nose point direction

Nose point direction calculation helps to determine head movement. Comparing with anchor points we can determine left, right, up and down head movements.

Anchor points (0,0) is set to nose point (31,34)

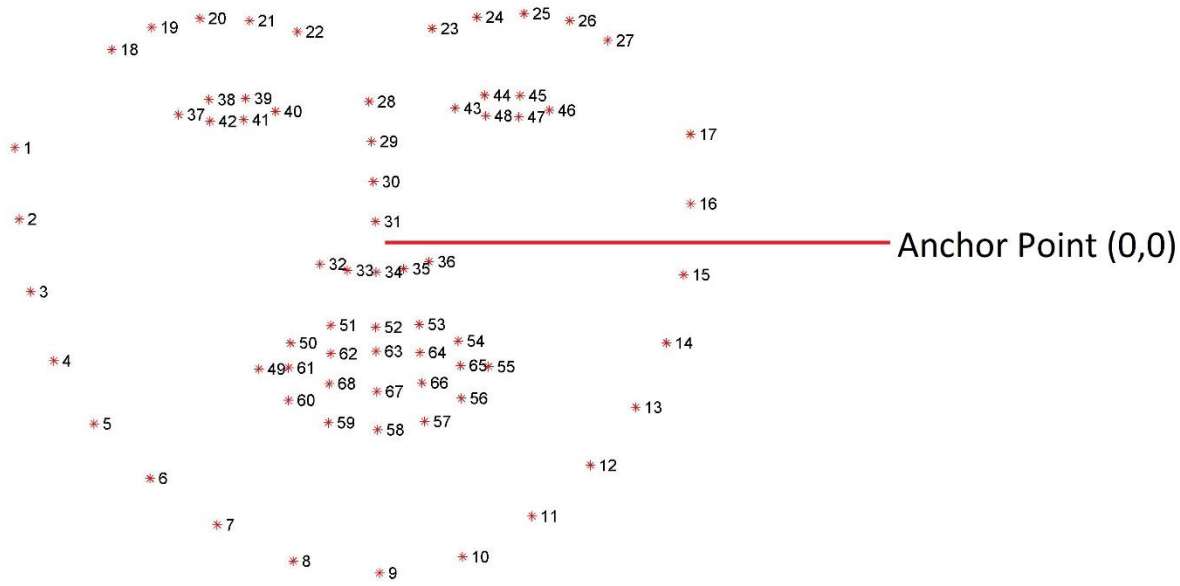


Fig 3.5: Comparison of nose point with anchor point

Right head movement = Nose point's X co-ordinate > anchor point's X co-ordinate

Left head movement = Nose point's X co-ordinate < anchor point's X co-ordinate

Up head movement = Nose point's Y co-ordinate > anchor point's Y co-ordinate

Down head movement = Nose point's Y co-ordinate < anchor point's Y co-ordinate

3.4 Summary

In this chapter, the theoretical part of our project has been described. We have tried to explain how the trained model is implemented based on facial landmark localization works on a face in detail. All the necessary equations and calculations that are needed has been shown.

Chapter 4
Structure of the System

4.1 Introduction

The structure of how the system works will be discussed in this chapter. Also, we will show the idea and implementation method of the interface. We have decided to build a human-computer interface. So, a set of explicit actions will be there. The workflow diagram of the system and details explanation with required technology will be added step by step.

4.2 Control Actions

Beginning with the first paragraph of the chapter, we want to propose a list of actions needed to trigger the mouse cursor for desired move. All these actions are facial movement based. No need of limb or other body muscle movement.

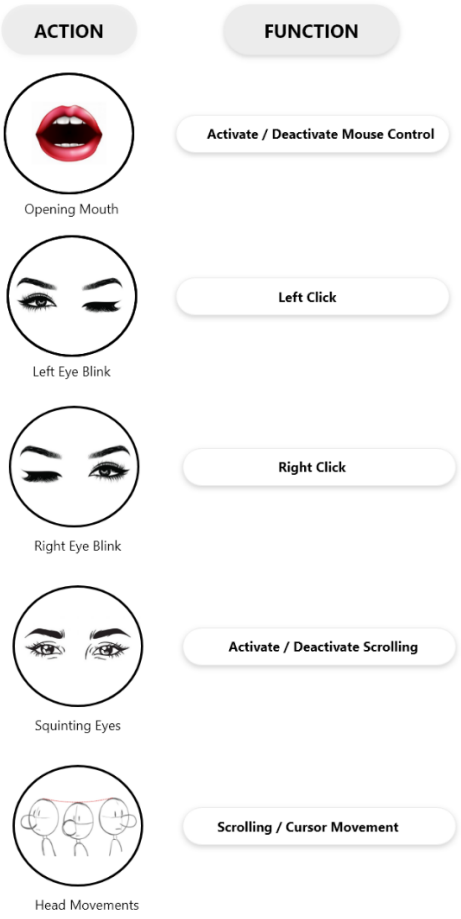


Fig 4.1: Control actions

Activate/Deactivate Mouse Control – Mouth Opening

Activation/Deactivation of mouse control is triggered by mouth opening. That means when the user wants to activate mouse for other actions, he or she will have to open their mouth. It is like grabbing the mouse with hand like an abled person. For deactivation, another mouth opening is needed.

Left Click – Left Eye Blink

The left eye blink for left click. It is anticipated and we want to design as a standard mouse.

Right Click – Right Eye Blink

The left eye blink for left click. It is anticipated and we want to design as a standard mouse.

Activate/Deactivate Scrolling – Squinting Eyes

Squinting (looking with partially closed eyes) will start/stop mouse scrolling.

Cursor Movement – Head Movement

Cursor will be moved with head movements.

4.2.1 Relation of Control Action with Theoretical Calculation

Control Action	Estimator
Activate/Deactivate Mouse	Mouth Aspect Ratio (MAR)
Left Click	Eye Aspect Ratio (EAR)
Right Click	Eye Aspect Ratio (EAR)
Activate/Deactivate Scrolling	Eye Aspect Ratio (EAR)
Cursor Movement	Nose Point Direction

Table 4.1: Control action estimator

We have proposed these actions considering the conveniences of the facial movements based on our real-life mouse operating scenarios. We hope this section of the interface will remain open for upgradation until final setup.

4.3 Workflow

This section will describe the sequence of events that result in any facial movement to a mouse cursor move.

4.3.1 Flowchart

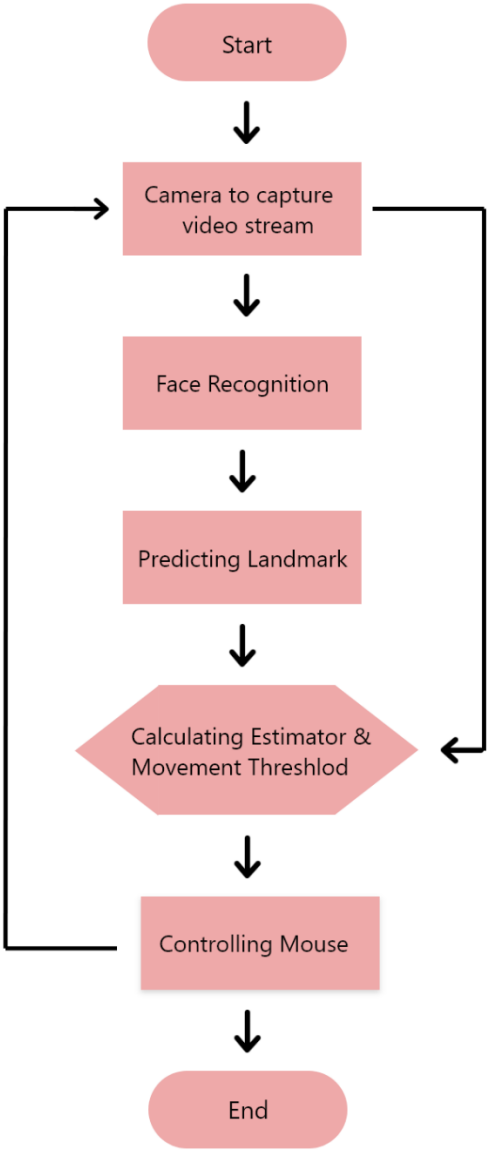


Fig 4.2: Flowchart of the system

4.4 Summary

In this chapter, we have described how the system works. The next chapter we will discuss about technologies and libraries are needed to implement the project.

Chapter 5
Technologies and Libraries

5.1 Introduction

In this chapter we will discuss relevant technologies and libraries required for our project. We are trying to develop an application for differently abled people to help them control a mouse of a computer with their facial movement. Our goal is to minimize the uses of hardware equipment. So, we opted for software model which only requires a web camera. We will talk over our used programming language, libraries and trained dataset.

5.2 Programming Language

Python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development [12]. Our supportive libraries, trained model all are compatible with python.

Version: 3.8



Fig 5.1: Python logo

5.3 Libraries

OpenCV

OpenCV (Open-Source Computer Vision Library), a library of programming functions mainly built aiming at real-time computer vision. This open-source computer vision and machine learning software library has more than 2500 algorithms which are optimized, which includes a set of both

classic and optimized computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos etc. [13]

Version: 4.5.1

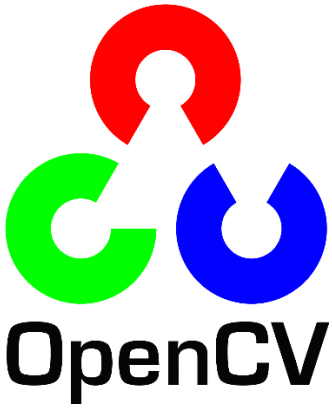


Fig 5.2: OpenCV logo

NumPy

NumPy is an open-source project aiming to enable numerical computing with Python Containing with large, multidimensional arrays and matrices it has also a large collection of high-level mathematical functions to operate this.

Version: v1.20.0



Fig 5.3: NumPy logo

Dlib

DLib is an open-source C++ library implementing a variety of machine learning algorithms, including classification, regression, clustering, data transformation, and structured prediction.

Version: 19.22.0



Fig 5.4: Dlib logo

5.4 Trained Model

Our trained model for this project is actually a facial shape predictor. The most common, well-known shape predictor is dlib's facial landmark predictor will be used to localize individual facial structures. The shape predictor algorithm implemented in the dlib library comes from Kazemi and Sullivan's paper. [5]

5.5 Dataset

The iBUG 300-W dataset was used to train the dlib shape predictor. The goal of iBUG-300W is to train a shape predictor which is capable of localizing each individual facial structure. The dataset consists of 68 pairs of integer values — these values are the (x, y)-coordinates of the facial structures depicted in Figure 3.1. To create the iBUG-300W dataset, researchers manually labeled each of the 68 coordinates on a total of 7,764 face images.

Link of the dataset: https://ibug.doc.ic.ac.uk/resources/300-W/

5.6 Summary

In this chapter, we reviewed the technology part, the next chapters will contain about technical implementation.

Chapter 6
Technical Module I: Detecting Facial Features

6.1 Introduction

In this chapter we are going to show how to work with images and videos to detect faces and facial parts such as eyes, nose, mouth, eyebrows and jaw. The prediction of 68 2D facial landmarks in static images and real time video sequence through camera was our initial steps of our system. We will also develop a utility function that helps us to calculate estimators and movement threshold of facial parts to control the mouse cursor. That will be discussed in another technical module later.

It is worth noting that this chapter work requires OpenCV library, dlib library, camera and image processing which has been already discussed in the previous chapter.

6.2 Static Images

Firstly, we have worked on images that our system would present, recognize the face, identify the facial parts and predict all the landmarks.

6.2.1 Loading and Presenting an Image

We have presented the image in an open window giving it a title “Front_Face” with OpenCV.



Fig 6.1: Loading an image

6.2.2 Face Recognition

We have recognized whether there is a face in the image. For this we have used dlib library's built-in function that returns a detector. The detector retrieves face information. But before doing this we had to convert the image into grayscale image as OpenCV could work fast. We know in image processing grayscale images reduce complexity comparing than RGB and color image. The face is shown inside a green rectangle.

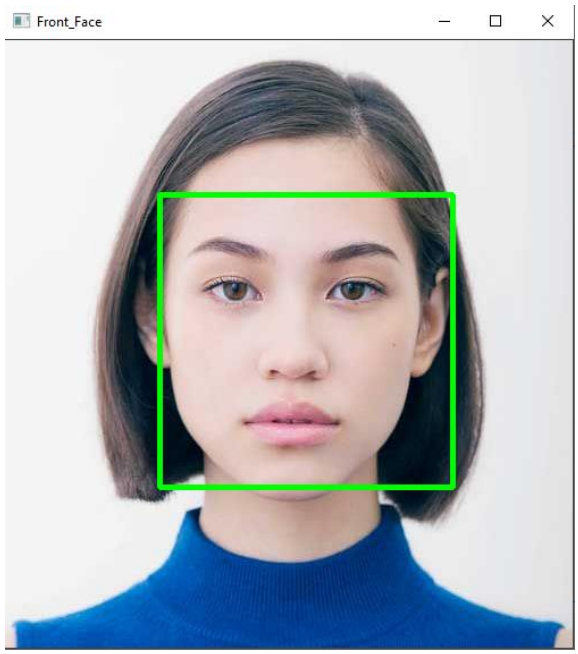


Fig 6.2: Face recognition

6.2.3 Identifying Facial Parts

We have identified mouth, nose, eyes, jaw, eyebrows after recognizing the face. Dlib library contain default shape_predictor() which works with several models. And it works with our trained model. The identified regions are marked with red dots.

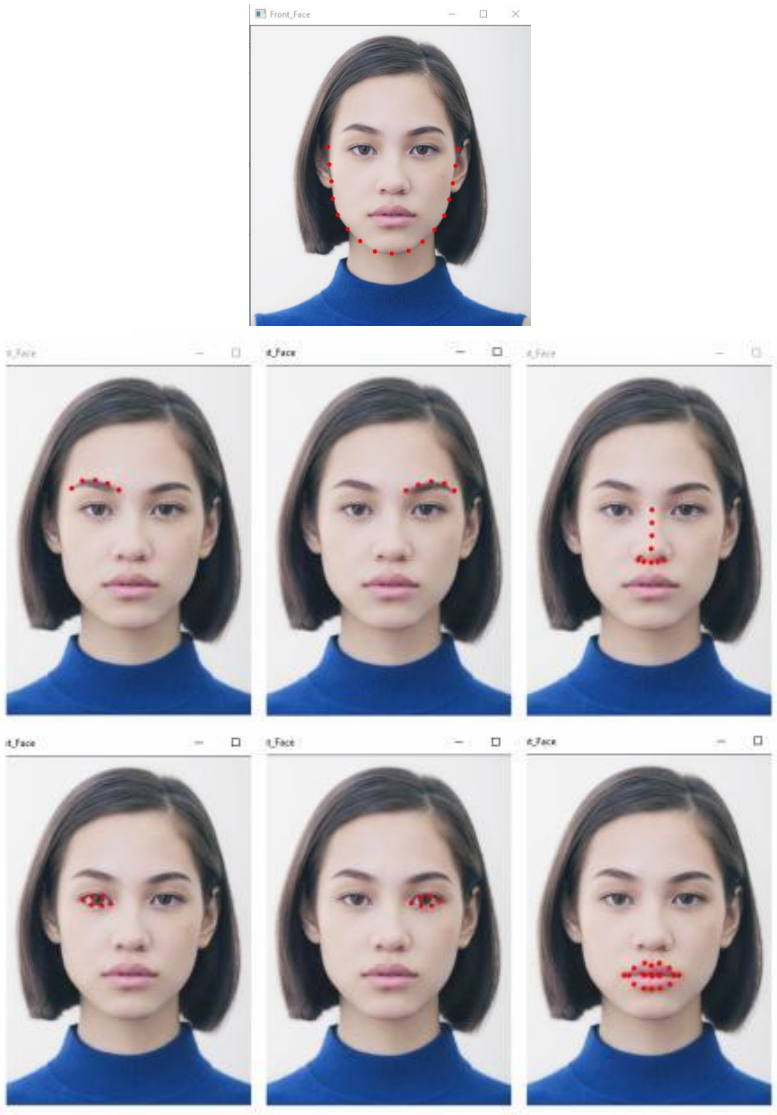


Fig 6.3: Facial parts localization

6.2.4 Predicting All Landmarks

The predictor function shows a frame that conveys a face with predicted 68 points to our diagram discussed in *Fig 3.1: 68 2D facial landmarks.*

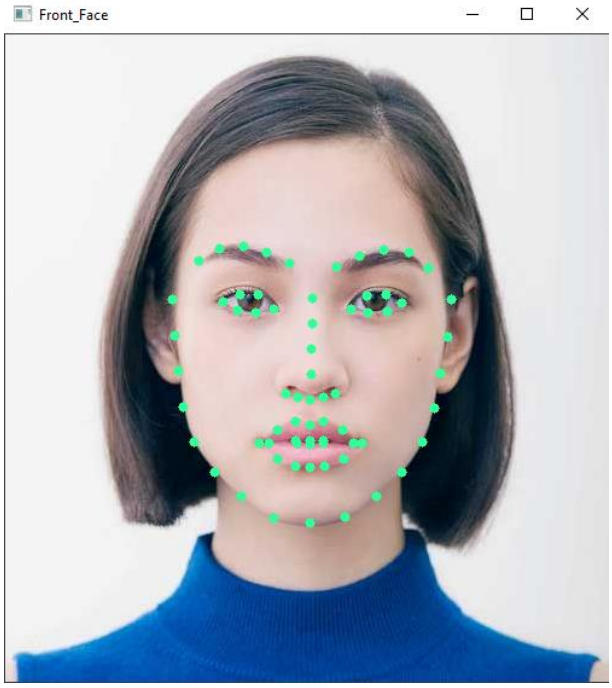


Fig 6.4: Predicting 68 landmarks on face

6.3 Real Time Video Sequence

We used webcam and did real time landmark prediction in video stream. We can do real-time facial landmarks prediction on faces by iterating through video frames with camera. The user will use a camera for computer operating. This will work as video frames for user. Here is the link for our tested real time video sequence:

https://drive.google.com/file/d/1YfC5cRrkgRU7SacqZ1P-vV0j6oM26VUf/view?usp=sharing

6.4 Challenges

One of the complex challenges was low light. As standard cameras or webcams are used it is difficult to find face and detecting the landmarks in low light. We present two portraits in two lighting condition and both of them works properly. Another problem we've faced is shown below. In this picture we can see the system did not locate the jaw properly as the picture was over processed or maybe the dataset trained in different facial shapes.

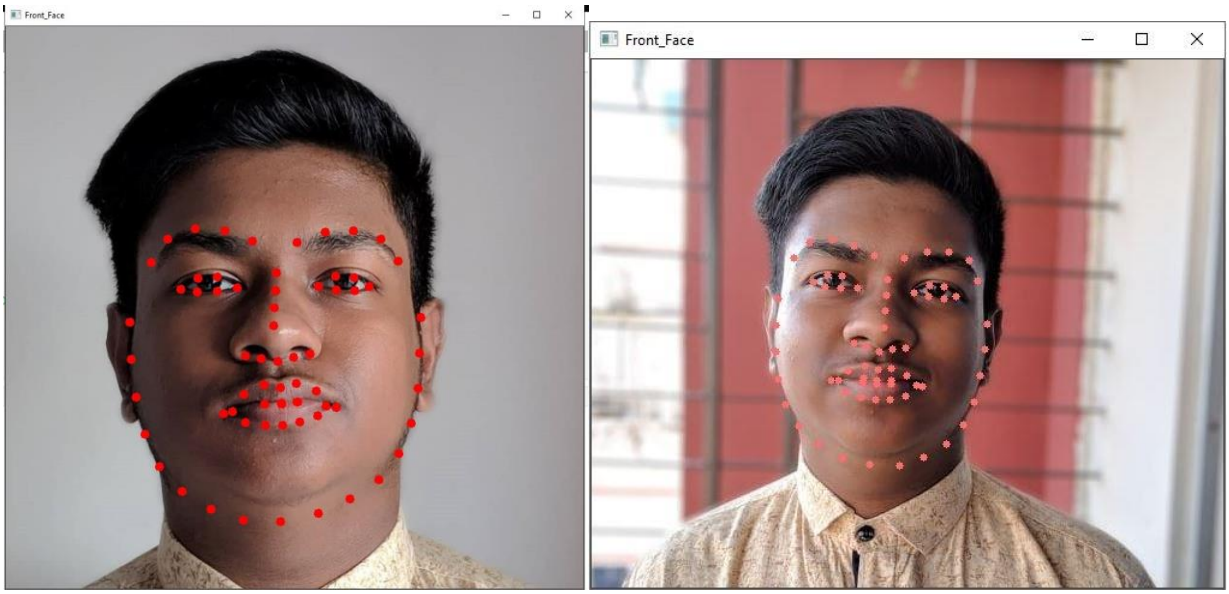


Fig 6.5: Two portraits in different light condition

In real time light might be an issue. Low light makes it challenging to find the face properly which we have to deal with.

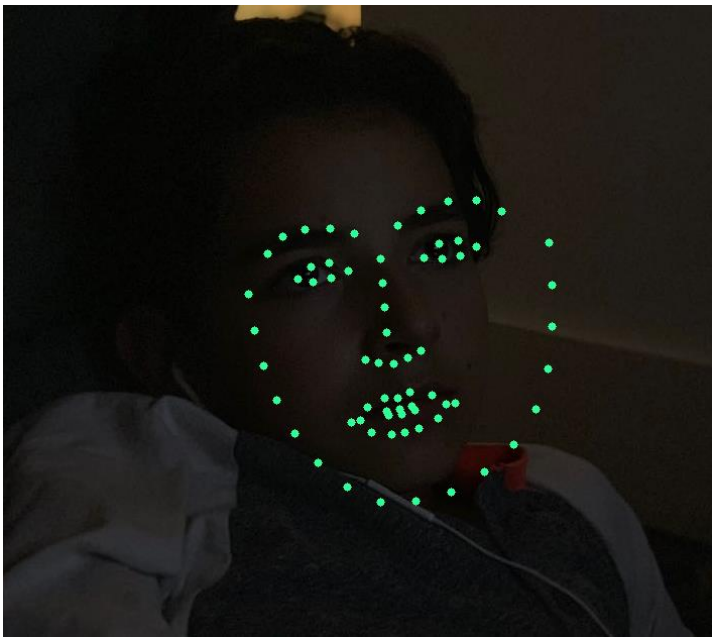
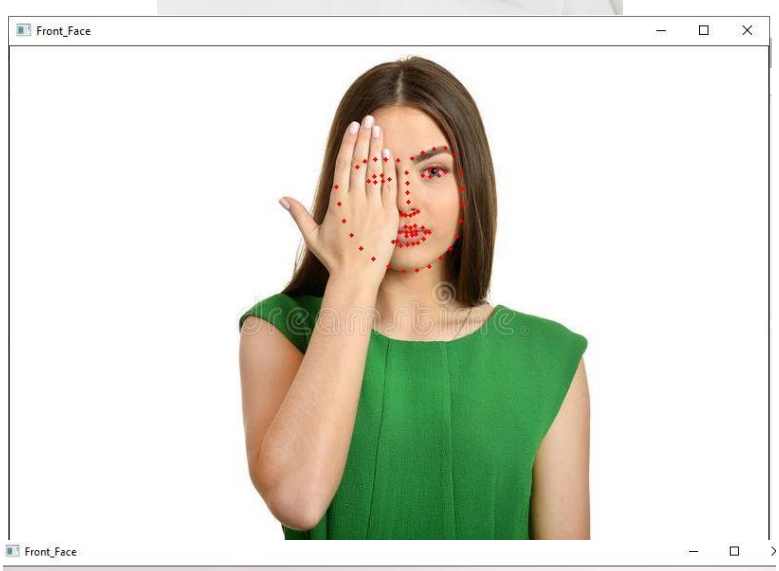
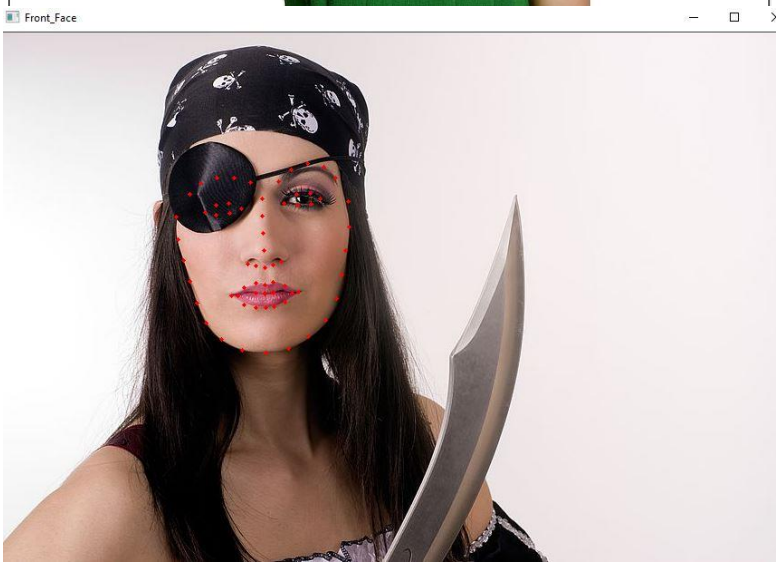


Fig 6.6: Low light detection problem

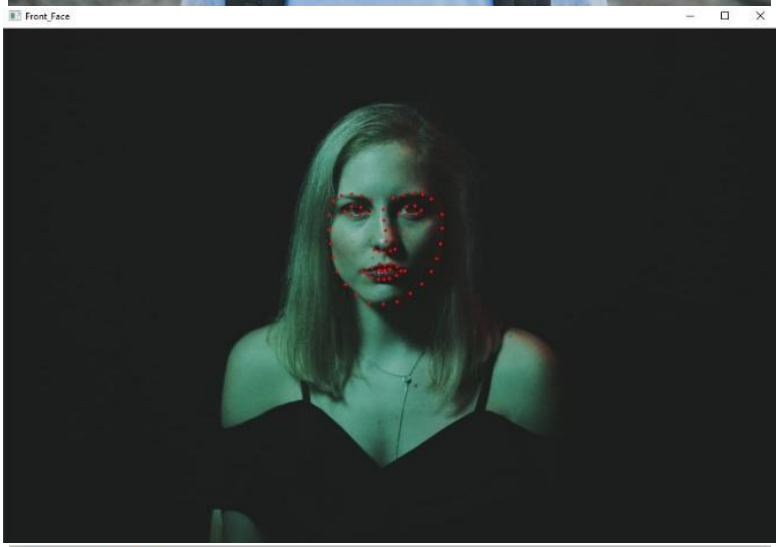
6.5 Testing Images

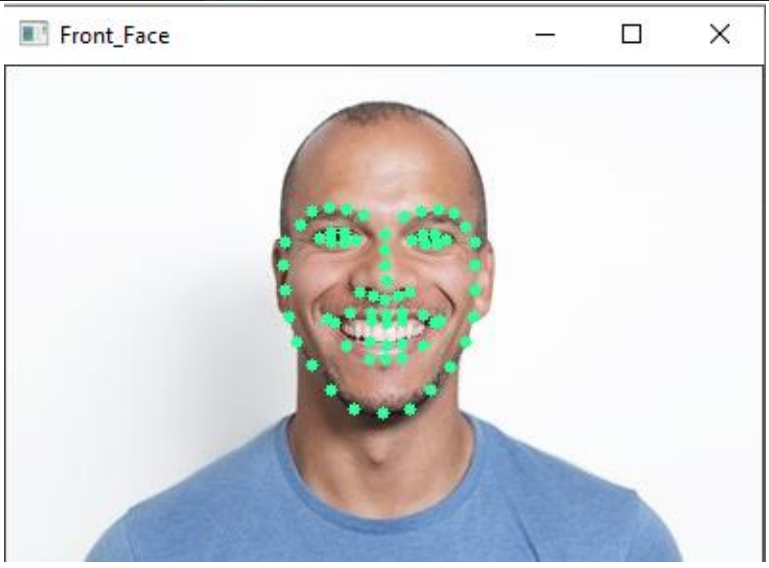









Front_Face


Front_Face

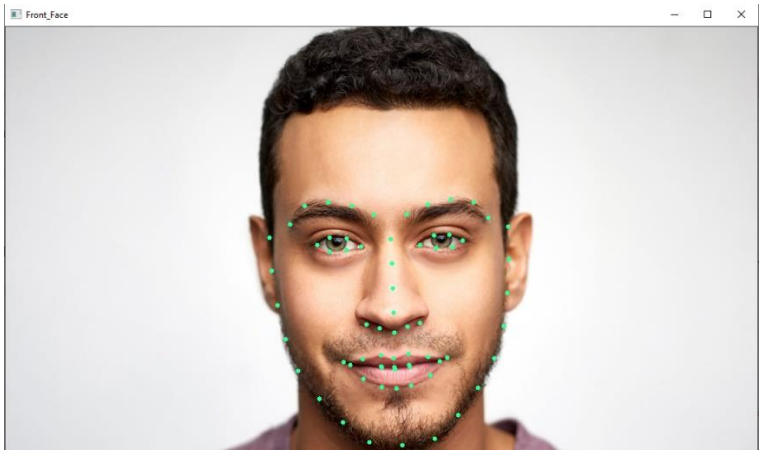


Fig 6.7: Test images

6.6 Summary

In this chapter we discussed about technical module I implementation.

Continued…

References

[1] H. OUANAN, M. OUANAN and B. AKSASSE, "Facial Landmark Localization: past, present and future," in *4th IEEE International Colloquium on Information Science and Technology (CIST)*, 2016.

[2] GlassOuse, "GlassOuse Assistive Device," 2020. [Online]. Available: https://glassouse.com/product/new-glassouse-v1-3-wireless-head-mouse-assistive-device-for-disabled-hands-free/.

[3] tobii dynavox, "tobii dynavox," tobii dynavox, 2021. [Online]. Available: https://www.tobiidynavox.com/devices/eye-gaze-devices/pceye-5-31ad2875/.

[4] AbleNet, "AbleNet," 2021. [Online]. Available: https://www.ablenetinc.com/trackerpro-2/.

[5] V. Kazemi and J. Sullivan, "One millisecond face alignment with an ensemble of regression trees,," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2014.

[6] T. Soukupova´ and J. Cˇech, "Real-Time Eye Blink Detection using Facial Landmarks," in *21st Computer Vision Winter Workshop*, 2016.

[7] C. Sagonas, G. Tzimiropoulos, S. Zafeiriou and M. Pantic, "300 Faces in-the-Wild Challenge: The first facial landmark localizationChallenge," 2013.

[8] N. Nikhade, M. Chaki, N. Shaikh and A.B.Lamgunde, "Mouse Handling Using Facial Features," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 4, no. 2, 2015.

[9] V. Nangare, U. Samant and S. Rabha, "Controlling Mouse Motions Using Eye Blinks, Head," *International Journal of Scientific and Research Publications Movements and Voice Recognition*, vol. 6, no. 3, 2016.

[10] Vijay Kotu, Bala Deshpande, "Ensemble Modeling," ScienceDirect, 2015. [Online]. Available: https://www.sciencedirect.com/topics/computer-science/ensemble-modeling.

[11] Q. CHENG, W. WANG, X. JIANG, S. HOU and Y. QIN, "Assessment of Driver Mental Fatigue," in *IEEE Access*, 2019.

[12] "What is Python? Executive Summary," Python, 2021. [Online]. Available: https://www.python.org/doc/essays/blurb/.

[13] K. Pulli, A. Baksheev, K. Kornyakov, and V. Eruhimov, "Realtime computer vision with OpenCV," *ACM Digital Library*, vol. 10, pp. 1-17, 2012.

Appendix A

```
#Loading Image

import cv2

# read the image

img = cv2.imread("face.jpg")

# show the image

cv2.imshow(winname="Front_Face", mat=img)

# Wait for a key press to exit

cv2.waitKey(delay=0)

# Close all windows

cv2.destroyAllWindows()

#Face Recognition

import cv2

import dlib

# Load the detector

detector = dlib.get_frontal_face_detector()

# read the image

img = cv2.imread("face.jpg")
```

```
    # Convert image into grayscale
    gray = cv2.cvtColor(src=img, code=cv2.COLOR_BGR2GRAY)
    # Use detector to find landmarks
    faces = detector(gray)
    for face in faces:
        x1 = face.left() # left point
        y1 = face.top() # top point
        x2 = face.right() # right point
        y2 = face.bottom() # bottom point
        # Draw a rectangle
        cv2.rectangle(img=img, pt1=(x1, y1), pt2=(x2, y2), color=(0, 255, 0),
thickness=4)
    # show the image
    cv2.imshow(winname="Front_Face", mat=img)
    # Wait for a key press to exit
    cv2.waitKey(delay=0)
    # Close all windows
cv2.destroyAllWindows()
    #Nose Point detection
```

```
import cv2

import dlib

# Load the detector

detector = dlib.get_frontal_face_detector()

# Load the predictor

predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

# read the image

img = cv2.imread("face.jpg")

# Convert image into grayscale

gray = cv2.cvtColor(src=img, code=cv2.COLOR_BGR2GRAY)

# Use detector to find landmarks

faces = detector(gray)

for face in faces:

    x1 = face.left() # left point

    y1 = face.top() # top point

    x2 = face.right() # right point

    y2 = face.bottom() # bottom point

    # Look for the landmarks

    landmarks = predictor(image=gray, box=face)
```

```
        x = landmarks.part(30).x
        y = landmarks.part(30).y
        # Draw a circle
        cv2.circle(img=img, center=(x, y), radius=5, color=(0, 255, 0), thickness=-1)
    # show the image
    cv2.imshow(winname="Front_Face", mat=img)
    # Wait for a key press to exit
    cv2.waitKey(delay=0)
    # Close all windows
cv2.destroyAllWindows()
    #Landmarks
    import cv2
    import numpy as np
    import dlib
    detector = dlib.get_frontal_face_detector()
    predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
    img = cv2.imread("face.jpg")
    gray = cv2.cvtColor(src=img, code=cv2.COLOR_BGR2GRAY)
    faces = detector(gray)
```

```
    for face in faces:
        x1 = face.left()
        y1 = face.top()
        x2 = face.right()
        y2 = face.bottom()

        landmarks = predictor(image=gray, box=face)

        for n in range(0, 68):
            x = landmarks.part(n).x
            y = landmarks.part(n).y

            cv2.circle(img=img, center=(x, y), radius=4, color=(153, 255, 51), thickness=-10)
    cv2.imshow(winname="Front_Face", mat=img)
    cv2.waitKey(delay=0)
cv2.destroyAllWindows()
    #Real Time
    import cv2
```

```
import dlib

detector = dlib.get_frontal_face_detector()

predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

cap = cv2.VideoCapture(0)

while True:
    _, frame = cap.read()
    gray = cv2.cvtColor(src=frame, code=cv2.COLOR_BGR2GRAY)

    faces = detector(gray)

    for face in faces:
        x1 = face.left()  # left point
        y1 = face.top()  # top point
        x2 = face.right()  # right point
        y2 = face.bottom()  # bottom point
```

```
        landmarks = predictor(image=gray, box=face)

        for n in range(0, 68):
            x = landmarks.part(n).x
            y = landmarks.part(n).y

            cv2.circle(img=frame, center=(x, y), radius=3, color=(0, 255, 0), thickness=-1)

        cv2.imshow(winname="Face", mat=frame)

        if cv2.waitKey(delay=1) == 27:
            break

    cap.release()

cv2.destroyAllWindows()
```